A digital twin for a lab robot

Test, acquire, simulate, evaluate — before touching the machine



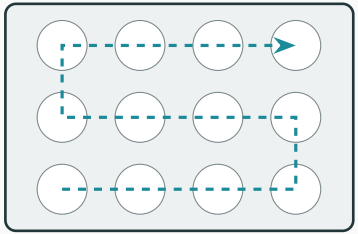
physical



digital

Lab automation loves things that don't change

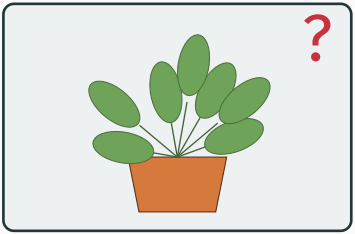
Well plate



Geometry known in advance

✔ write the path once, run it forever

Living sample

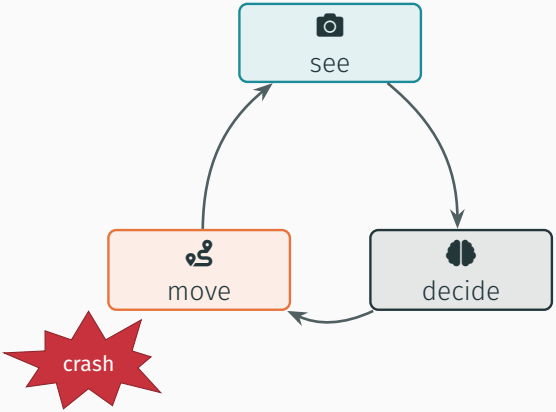


Grows, moves, differs every time

✖ no path can be written in advance

Most self-driving labs require every object to be *defined and hard-coded before the run*.
Plants, duckweed, tissues don't fit that paradigm.

Vision fixes adaptability … and creates a safety problem

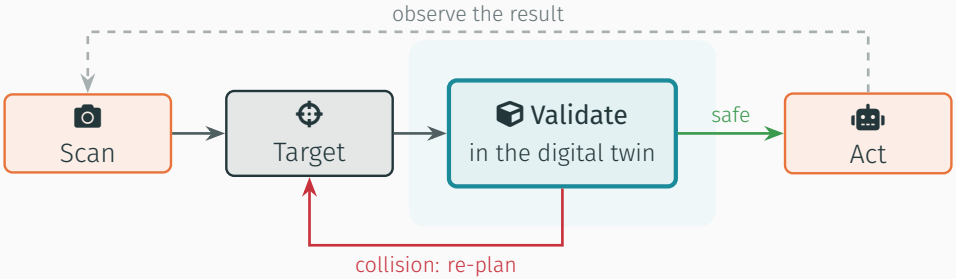


(A) Adaptability. The robot looks at the sample and decides where to go.

(B) Safety. A robot that decides its own moves can also decide to hit the sample, the labware, or itself — moves nobody reviewed.

That's why vision-driven robots are usually kept away from living samples.

The idea: every move goes through the twin first

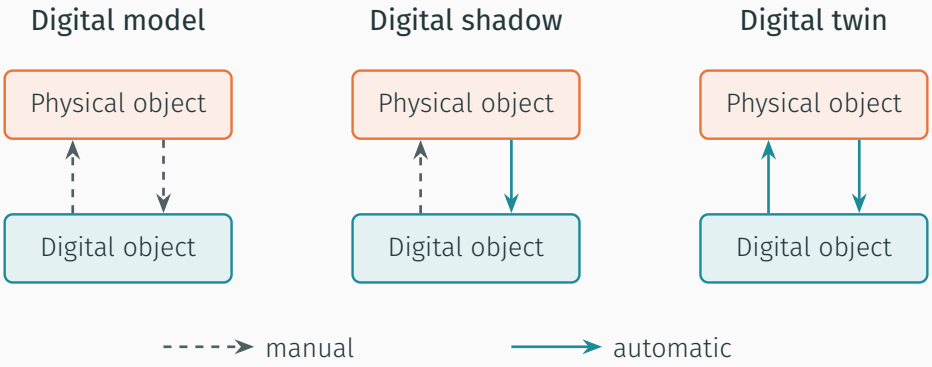


Physical
the real Jubilee and its camera

Digital
a synchronized 3D copy in Blender

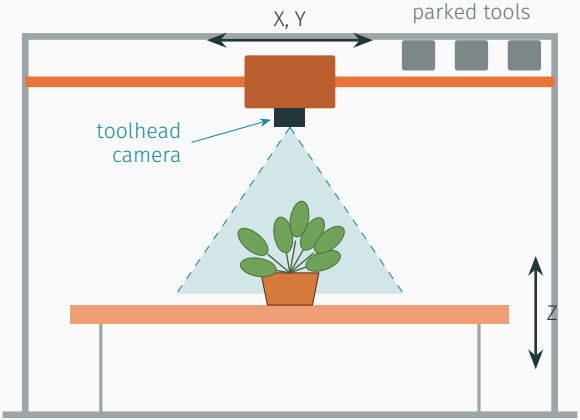
Rule
nothing reaches the motors without passing the twin

Model, shadow, twin: it's about which way the data flows



A twin is *kept in sync* with the machine (positions, tools, calibration) and *talks back* to it (which moves are allowed).

The machine: Jubilee, with a camera on the toolhead



- Open-source **tool-changing** motion platform
- **X**/**Y** move the toolhead (and the camera)
- **Z** moves the *bed*: bigger Z = bed lower = sample further from the camera
- Tools: camera, inoculator, pen, fluorescence probe…

You'll need the Z convention in Exercise 1.

One machine, two copies

real toolhead camera



same calibration

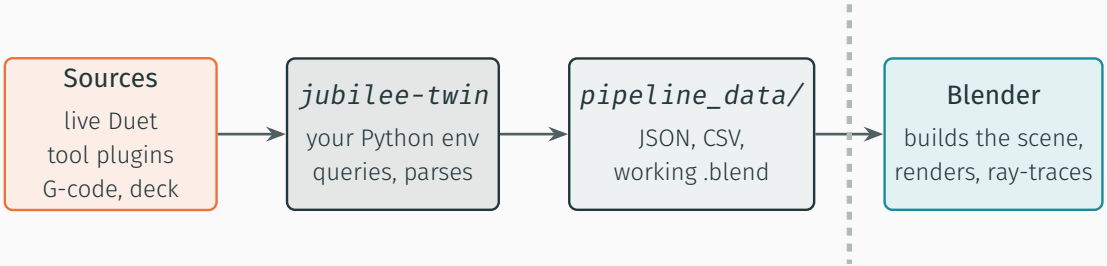
`Toolhead_Cam` in Blender



Geometry from the machine: tool slots, park positions, offsets and head position are read from the Duet (or a saved snapshot), so the scene matches the real deck.

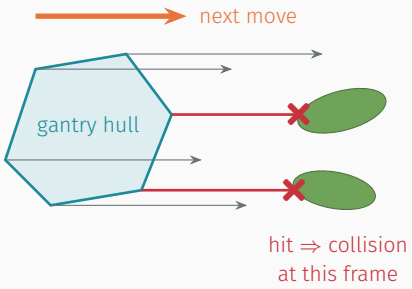
Camera from calibration: the pinhole intrinsics f_x, f_y, c_x, c_y become Blender's lens and shift, so a pixel in the twin means the same thing as a pixel on the machine.

How the twin is wired



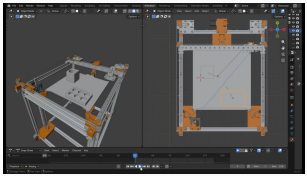
Every number in the scene comes from a file you can open — and a trace page says which source won.

The collision gate: rays along the motion



For each frame of the planned motion:

1. take the convex hull of the moving parts (+ active tool)
2. cast rays from its vertices, along the motion
3. any scene object hit = collision, with frame and object name



What a twin unlocks

Ex. 1

Test before running

Find the crash in the twin instead of on the machine

Ex. 2

Acquire virtually

Script and debug an acquisition that makes its own decisions

Ex. 3

Synthetic data

Unlimited images with labels (depth, masks, poses) for free

Ex. 4

An exact yardstick

Score 3D vision models against geometry we know exactly

Unlock 1 — test before running



What the toolhead camera sees
a moment before a crash.

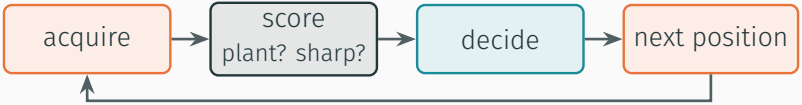
An old plan, written for a flat Petri dish. Today there's a basil plant on the bed.

In the twin you can:

- replay the G-code and get every colliding frame
- ask questions you'd never ask the real machine: *what's the lowest Z that's still safe?*

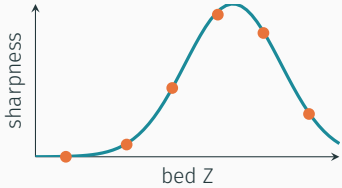
A crash in the twin costs about 15 seconds.

Unlock 2 — acquisitions that decide





bed too high | closer | in focus



The virtual camera has a real depth of field, so autofocus is a genuine decision.

Unlock 3 — same geometry, many appearances



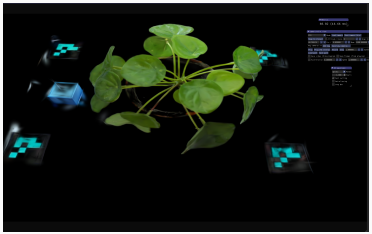


K, camera pose, full 3D mesh

An **HDRI** is a 360° photo that both lights the scene and is the background. Swap it and the appearance changes, but the annotations don't.

Unlock 4 — a yardstick for 3D vision

Real plant



Gaussian-splatting reconstruction. It looks right, but *how* right? No ground truth exists.

Twin plant



MoGe-2
one image

ray casting
exact



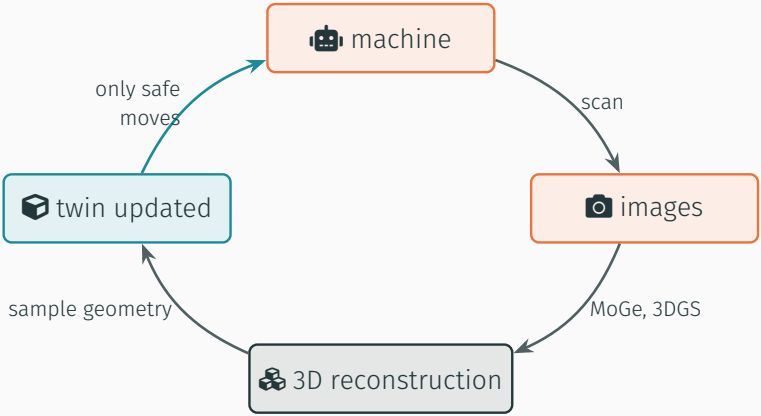
prediction

true surface

nearest distance

Same camera, same image, so the clouds are directly comparable. Distances to the true surface give **scale error**, **accuracy**, **completeness** and **Chamfer distance**.

Where this is going: closing the loop



Today the plant is a hand-made model. Next: the twin learns the sample from the machine's own images, and Unlock 4 tells us how much margin that requires.

What the twin does *not* know

Only what's in it

- A plant that grew since the scene was built is invisible.
- Ground truth is only as good as the mesh.

Sim-to-real gap

- Pinhole render: lens distortion is recorded, not rendered.
- Materials and lighting are approximations.

The collision check

- Translations only: no rotating or deforming parts.
- Rays along the motion: a very fast step can tunnel through a thin object.
- Its answer is "no hit found", not a proof.

Use it as a gate, not a guarantee: keep a margin.

Today

	Exercise	You will…	Time
0	Setup	build the scene, take a virtual picture	10 min
1	Check before running	find and fix a crash in a scan plan	25 min
2	Virtual camera	raster, pick a tile, autofocus	25 min
3	Lighting & background	swap HDRIs in Blender and in Python	20 min
4	MoGe-2	reconstruct from one image, score it	35 min

Notebook: `jubilee-blender-twin/workshop/digital_twin_workshop.ipynb`

Needs Blender 5 on `PATH` and `jubilee-twin setup-scene` run once.